DISEÑO DE COMPILADORES



Capítulo 5

Análisis Semántico

Observación. *El presente documento sirve de complemento al contenido que referencia y no se constituye en el detalle del mismo, ya que conceptos y contenidos pueden haber sido desarrollados en forma presencial, virtual o asignado como ejercicio. Ante cualquier duda se recomienda consultar con el profesor a cargo de la cátedra.*

Profesor: Sergio A. Aranda

ANALISIS SEMÁNTICO

Definición dirigida por la sintaxis

Una Def. Dirigida por la Sintaxis es una gramática libre de contexto, formada por atributos y reglas o acciones. Los atributos se asocian con los símbolos gramaticales y las reglas con las producciones.

Si X es un símbolo y a un atributo, escribimos X.a

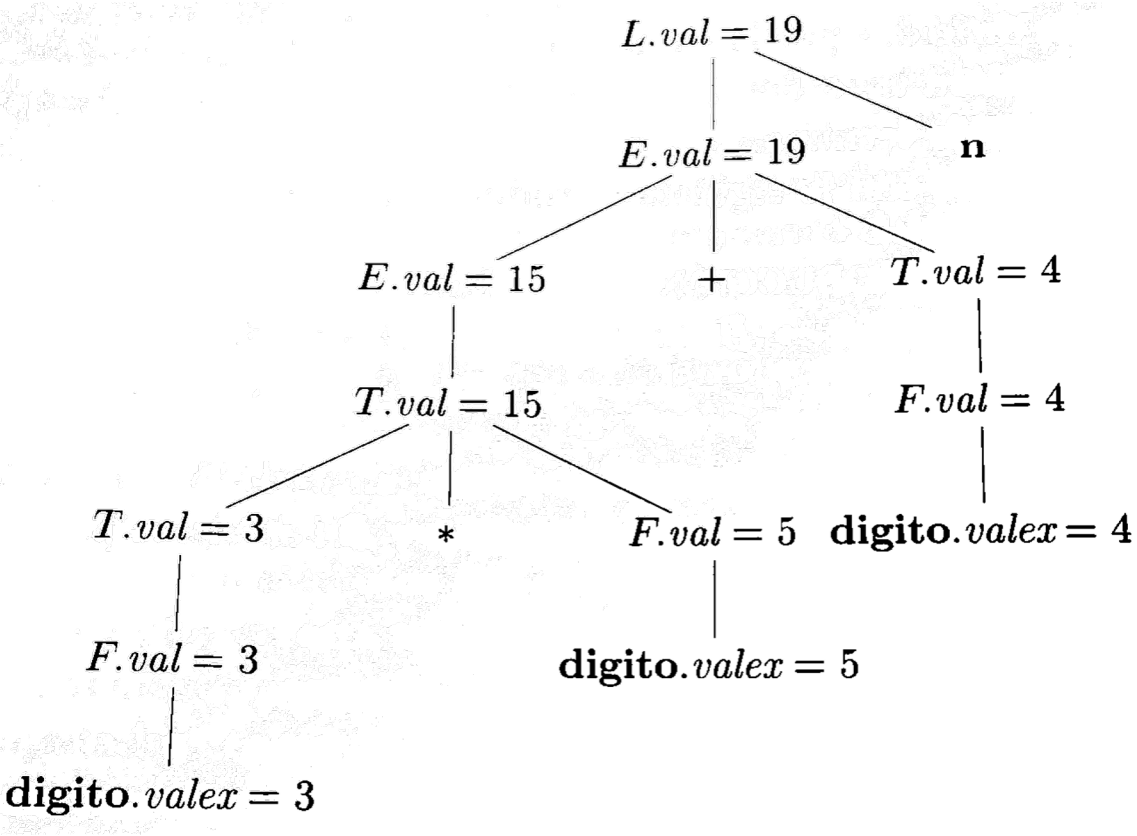
TIPOS DE ATRIBUTOS

a. **Sintetizados**, cuyo valor depende de sus hijos y del nodo mismo donde se encuentra.
b. **Heredados**, cuyo valor depende de valores en el padre, en el nodo mismo y en sus hermanos.

Ejemplo: Expresiones aritméticas para los operadores + y *

	PRODUCCIÓN	REGLAS SEMÁNTICAS
1)	$L \rightarrow E\ \mathbf{n}$	$L.val = E.val$
2)	$E \rightarrow E_1\ +\ T$	$E.val = E_1.val + T.val$
3)	$E \rightarrow T$	$E.val = T.val$
4)	$T \rightarrow T_1\ *\ F$	$T.val = T_1.val \times F.val$
5)	$T \rightarrow F$	$T.val = F.val$
6)	$F \rightarrow (\ E\ )$	$F.val = E.val$
7)	$F \rightarrow \mathbf{digito}$	$F.val = \mathbf{digito}.\text{lexval}$

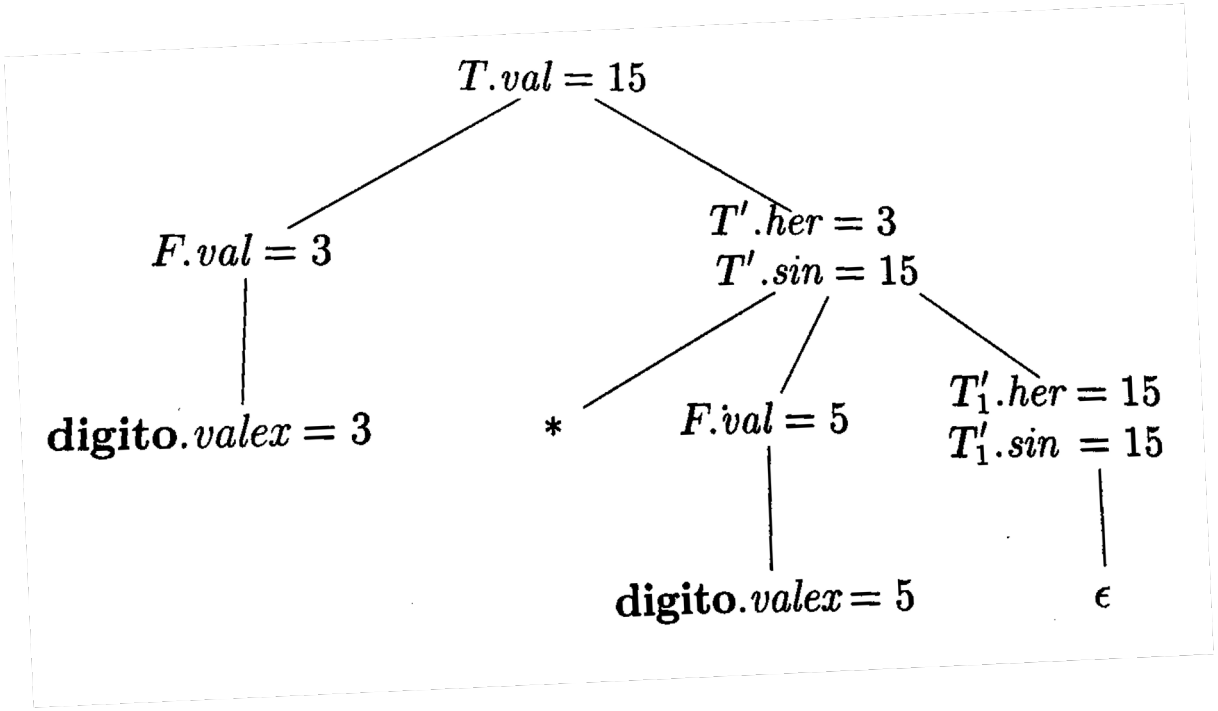
Ejemplo: Expresiones aritméticas para los operadores + y *



Ejemplo: Atributos sintetizados y heredados

	PRODUCCIÓN	REGLAS SEMÁNTICAS
1)	$T \rightarrow F\ T'$	$T'.her = F.val$ $T.val = T'.sin$
2)	$T' \rightarrow *\ F\ T'_1$	$T'_1.her = T'.her \times F.val$ $T'.sin = T'_1.sin$
3)	$T' \rightarrow \epsilon$	$T'.sin = T'.her$
4)	$F \rightarrow \mathbf{digito}$	$F.val = \mathbf{digito}.valex$

Ejemplo: Expresiones aritméticas para los operadores + y *



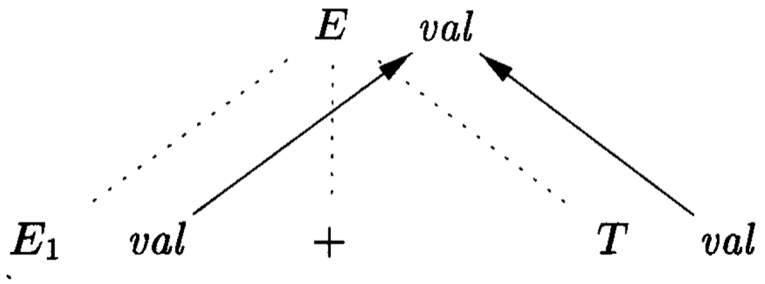
ANALISIS SEMÁNTICO

Gráficos de dependencias

Un grafo de dependencia describe el flujo de información entre los atributos en un árbol de análisis sintáctico. Una flecha de una instancia de atributos a otra significa que el valor de la primera es necesario para calcular el de la segunda.

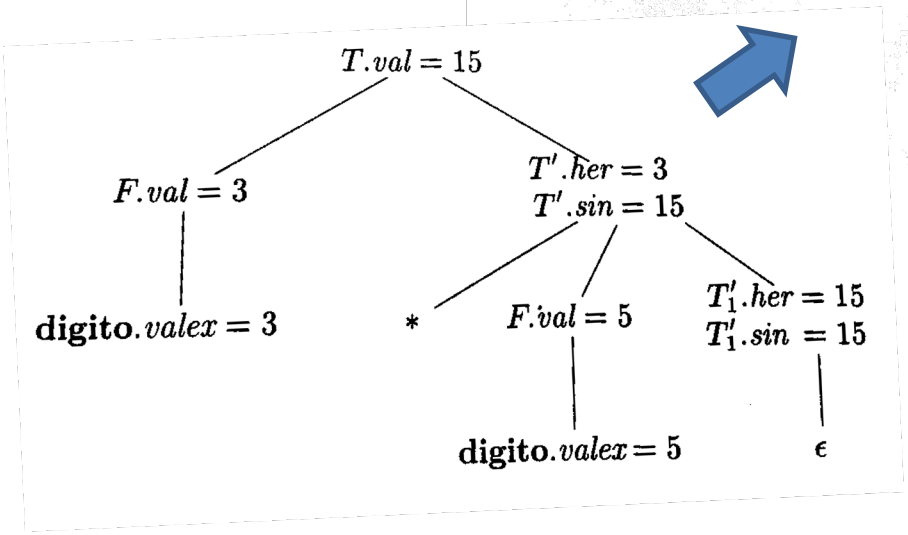
- Para cada nodo del árbol de análisis sintáctico, etiquetado por el símbolo gramatical X, el grafo tiene un nodo para cada atributo asociado con X.

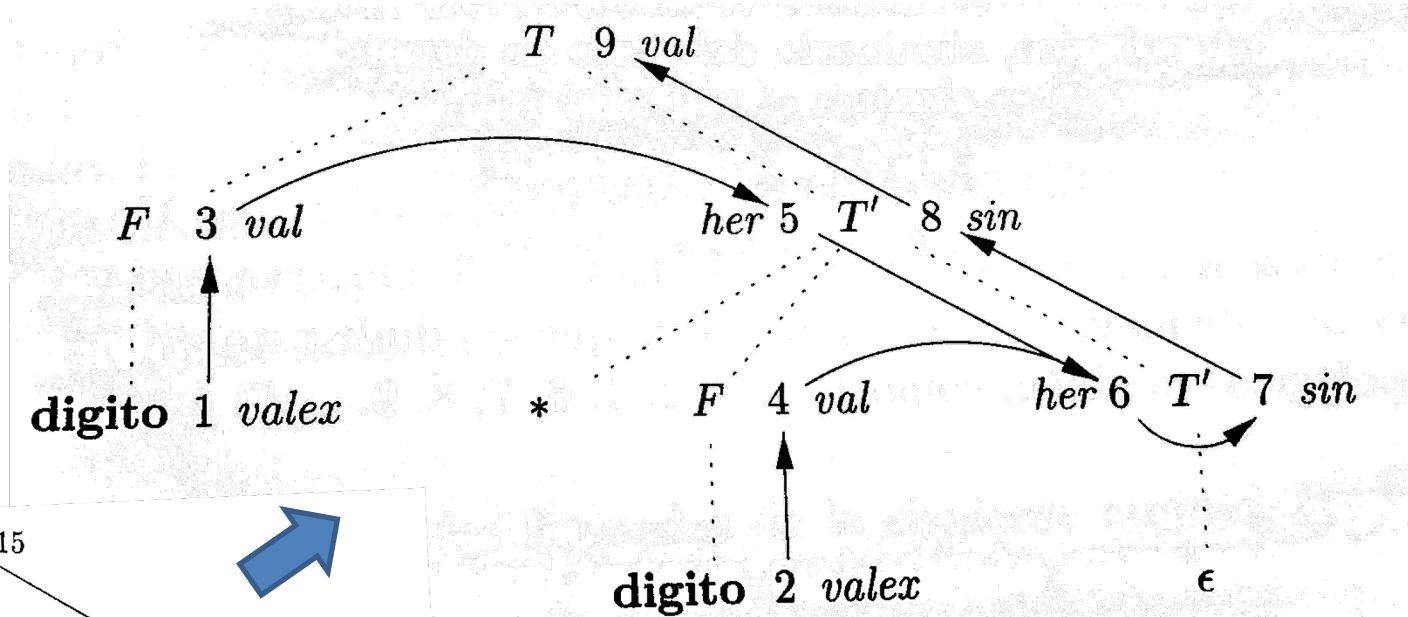
PRODUCCIÓN	REGLA SEMÁNTICA
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$



ANALISIS SEMÁNTICO

Gráficos de dependencias





	PRODUCCIÓN	REGLAS SEMÁNTICAS
1)	$E \rightarrow E_1 + T$	$E.nodo = \textbf{new}\ Nodo('+', E_1.nodo, T.nodo)$
2)	$E \rightarrow E_1 - T$	$E.nodo = \textbf{new}\ Nodo('-', E_1.nodo, T.nodo)$
3)	$E \rightarrow T$	$E.nodo = T.nodo$
4)	$T \rightarrow (E)$	$T.nodo = E.nodo$
5)	$T \rightarrow \textbf{id}$	$T.nodo = \textbf{new}\ Hoja(\textbf{id}, \textbf{id}.entrada)$
6)	$T \rightarrow \textbf{num}$	$T.nodo = \textbf{new}\ Hoja(\textbf{num}, \textbf{num}.val)$

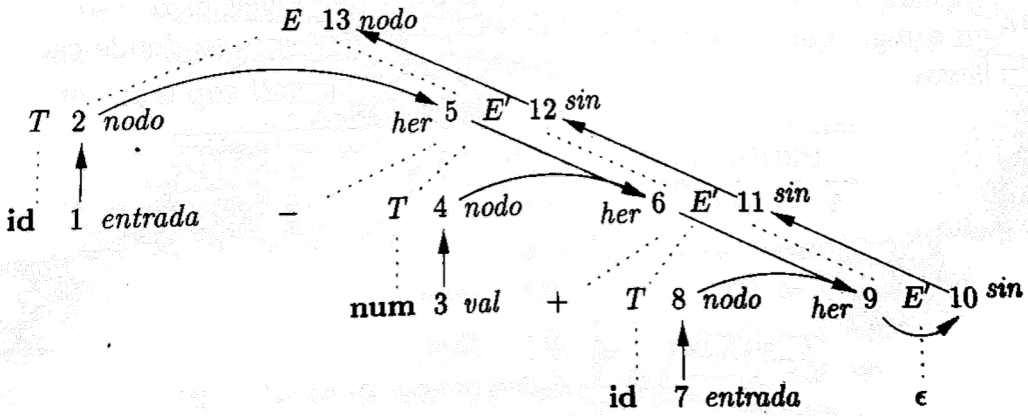
Construir el árbol sintáctico para: a – 4 + c

ANALISIS SEMÁNTICO

EJEMPLO (Atributos sintetizados)

	PRODUCCIÓN	REGLAS SEMÁNTICAS
1)	$E \rightarrow T\ E'$	$E.nodo = E'.sin$
		$E'.her = T.nodo$
2)	$E' \rightarrow +\ T\ E'_1$	$E'_1.her =$ **new** $Nodo('+', E'.her, T.nodo)$
		$E'.sin = E'_1.sin$
3)	$E' \rightarrow -\ T\ E'_1$	$E'_1.her =$ **new** $nodo('-', E'.her, T.nodo)$
		$E'.sin = E'_1.sin$
4)	$E' \rightarrow \epsilon$	$E'.sin = E'.her$
5)	$T \rightarrow (\ E\ )$	$T.nodo = E.nodo$
6)	$T \rightarrow$ **id**	$T.nodo =$ **new** $Hoja($**id**, **id**$.entrada)$
7)	$T \rightarrow$ **num**	$T.nodo =$ **new** $Hoja($**num**, **num**$.entrada)$

Construcción de árboles sintácticos durante el análisis sintáctico descendent



Escribir un Traductor Dirigido por la Sintaxis para una gramática que acepta cadenas de operadores asociativos por la izquierda con el siguiente alfabeto Σ={ +, -}.

$E \rightarrow E_1 + T$	$\{ E.val := E_1 val + T.val \}$
$E \rightarrow E_1 - T$	$\{ E.val := E_1 val - T.val \}$
$E \rightarrow T$	$\{ E.val := T.val \}$
$T \rightarrow (E)$	$\{ T.val := E.val \}$
$T \rightarrow$ **núm**	$\{ T.val :=$ **núm**$.val \}$

ANALISIS SEMÁNTICO
TRANSFORMACIÓN DE ACCIONES SEMÁNTICAS

EJERCICIO: Considerando la gramática anterior, se pide:

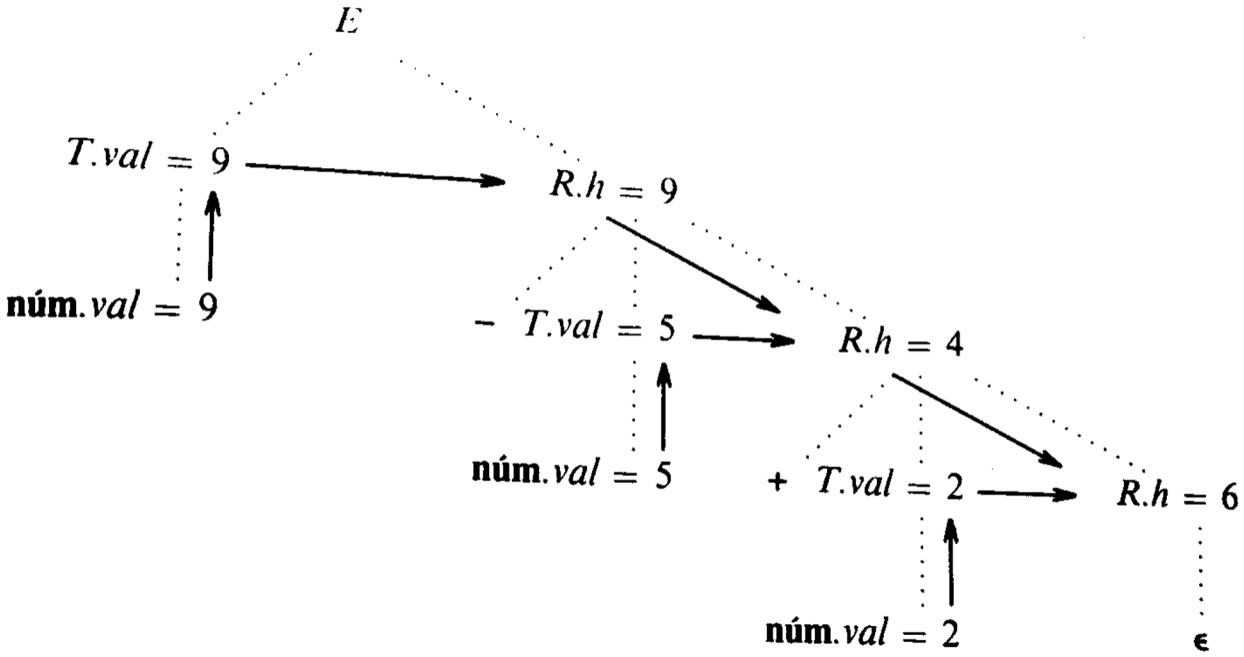
1. Obtener el árbol sintáctico para la cadena 9 – 5 + 2
2. Eliminar la recursión por la izquierda
3. Ordenar las reglas semánticas
4. Obtener el grafo de dependencias para la misma cadena de entrada

Gramática de operadores asociativos por la izquierda con recursión por la izquierda eliminada.

$$
\begin{array}{lll}
E \rightarrow T & \{ R.h & := T.val \} \\
\quad R & \{ E.val & := R.s \} \\
R \rightarrow + & & \\
\quad T & \{ R_1.h & := R.h + T.val \} \\
\quad R_1 & \{ R.s & := R_1.s \} \\
R \rightarrow - & & \\
\quad T & \{ R_1.h & := R.h - T.val \} \\
\quad R_1 & \{ R.s & := R_1.s \} \\
R \rightarrow \epsilon & \{ R.s & := R.h \}
\end{array}
$$

$$
\begin{array}{lll}
T \rightarrow (& & \\
\quad E & & \\
\quad) & \{ T.val & := E.val \} \\
T \rightarrow \textbf{núm} & \{ T.val & := \textbf{núm}.val \}
\end{array}
$$

El grafo de dependencias para la gramática de operadores binarios asociativos por la izquierda, sin recursión por la izquierda para la cadena 9 – 5 + 2 queda:



El método general para adaptar otros esquemas de traducción dirigidas por la sintaxis es el sgte:

a. Suponer que se cuenta con

$$A \rightarrow A_1\ Y \qquad \{ A.a := g(A_1.a,\ Y.y) \}$$
$$A \rightarrow X \qquad \{ A.a := f(X.x) \}$$

b. El algoritmo para eliminar la recursión por la izquierda simple es:

$$A \rightarrow X\ R$$
$$R \rightarrow Y\ R \mid \epsilon$$

c. Aplicar:

$$A \rightarrow X \quad \{ R.h := f(X.x) \}$$
$$\quad R \quad \{ A.a := R.s \}$$
$$R \rightarrow Y \quad \{ R_1.h := g(R.h,\ Y.y) \}$$
$$\quad R_1 \quad \{ R.s := R_1.s \}$$
$$R \rightarrow \epsilon \quad \{ R.s := R.h \}$$

$A \rightarrow A_1\ Y \quad \{ A.a := g(A_1.a, Y.y) \}$

$A \rightarrow X \quad \{ A.a := f(X.x) \}$

	PRODUCCIÓN	REGLAS SEMÁNTICAS
1)	$E \rightarrow E_1 + T$	$E.nodo =$ **new** $Nodo('+', E_1.nodo, T.nodo)$
2)	$E \rightarrow E_1 - T$	$E.nodo =$ **new** $Nodo('-', E_1.nodo, T.nodo)$
3)	$E \rightarrow T$	$E.nodo = T.nodo$
4)	$T \rightarrow (E)$	$T.nodo = E.nodo$
5)	$T \rightarrow$ **id**	$T.nodo =$ **new** $Hoja($**id**, **id**$.entrada)$
6)	$T \rightarrow$ **num**	$T.nodo =$ **new** $Hoja($**num**, **num**$.val)$

$A \rightarrow X \quad \{ R.h := f(X.x) \}$

$R \quad \{ A.a := R.s \}$

$R \rightarrow Y \quad \{ R_1.h := g(R.h, Y.y) \}$

$R_1 \quad \{ R.s := R_1.s \}$

$R \rightarrow \epsilon \quad \{ R.s := R.h \}$

E -> T {$R._h = T._{nodo}$ }
R {$E._{nodo} = R._s$ }

R -> - T { R1.h = newNodo(´-´, R.h, T.nodo) }
R1 {R.s = R1.s }

R -> + T {R1.h = newNodo(´+´, R.h, T.nodo) }
R1 { R.s = R1.s}

R -> € { R.s= R.h }

TABLA DE SÍMBOLOS
HASHING - ESTRUCTURA

```
#define MAXBUCKET    12
#define SLOTS        2

//Estructura de datos - Tabla de elementos
typedef struct elem{
    int id;
    int deleted;
    // otros datos;
} element;

element hashTable[MAXBUCKET][SLOTS];

// Hash Function
int hash( int id )
{
    return( id%MAXBUCKET );
}
```

```
// Inicializa la tabla en cero
void initTable( element a[][SLOTS] )
{
        int i,j;
        for(i=0; i<MAXBUCKET; i++)
                for(j=0;j<SLOTS;j++){
                        a[i][j].id=0;
                        a[i][j].deleted=0;
                }
}
```

TABLA DE SÍMBOLOS
HASHING – BÚSQUEDA

```
int searchElement( element a[][SLOTS], int n )
{
    int homebucket, i, j;

    homebucket=hash( n );
    for( i = homebucket; ; ){
        for(j=0; j<SLOTS; j++){
            if( a[i][j].id == n  && !(a[i][j].deleted) ){
                printf("\n Elemento %d se encontro en la posición [%d][%d]",n,i,j);
                return 1;
            }
            else if( ( a[i][j].id == 0 && !(a[i][j].deleted) ) ){
                printf("\n Elemento %d NO se encontro en la tabla.",n);
                return 0;
            }
        }
        i++;
        if( i == MAXBUCKET )
            i=0;
        if( i == homebucket ){
            printf("\n Elemento %d NO se encontro en la tabla.",n);
            return 0;
        }
    }
    return 0;
}
```

TABLA DE SÍMBOLOS

HASHING – IMPRIMIR LA TABLA

```
// Imprime la tabla Hash
void printHashTable( element a[][SLOTS] )
{
    int i,j;

    printf("\n");
    printf(" #  ");
    for(j=0; j<SLOTS; j++)
        printf("  S%2d ",j);

    printf("\n");

    for( i=0; i < MAXBUCKET; i++ ){
            printf("%3d:",i);
            for(j=0; j< SLOTS; j++){
                if( a[i][j].id != 0 && !(a[i][j].deleted) )
                    printf("%6d", a[i][j].id);
                else
                    //printf("      ");
                    printf("%6d,(%d)", a[i][j].id, a[i][j].deleted);
            }
            printf("\n");
    }

}
```

TABLA DE SÍMBOLOS

HASHING – INSERTAR ELEMENTOS

```
// Agregar un elemento. Estrategia de colisión, linear Probing
int insertElement( element a[][SLOTS], int n )
{
    int homebucket, i, j;

    homebucket=hash( n );

    for( i = homebucket; ; ){
        for(j=0; j<SLOTS; j++){
            if( a[i][j].id == 0 || a[i][j].deleted ){
                a[i][j].id= n;
                a[i][j].deleted= 0;
                printf("\n Elemento %d se inserto en la posición [%d][%d]",n,i,j);
                return 1;
            }
        }
        i++;
        if( i == MAXBUCKET )
            i=0;
        if( i == homebucket ){
            printf("\n Error: La tabla esta llena.");
            return 0;
        }
    }
    return 0;
}
```

TABLA DE SÍMBOLOS

HASHING – ELIMINAR UN ELEMENTO

```
int deleteElement( element a[][SLOTS], int n)
{
    int homebucket, i, j;

    homebucket=hash( n );
    for( i = homebucket; ; ){
        for(j=0; j<SLOTS; j++){
            if( a[i][j].id == n  && !(a[i][j].deleted) ){
                a[i][j].deleted= 1;
                printf("\n Elemento %d se elimino en la posicion [%d][%d]",n,i,j);
                return 1;
            }
            else if( a[i][j].id == 0 && !( a[i][j].deleted) ){
                printf("\n Error %d no se encuentra en la tabla.", n);
                return 0;
            }
        }
        i++;
        if( i == MAXBUCKET )
            i=0;
        if( i == homebucket ){
            printf("\n Error %d no se encuentra en la tabla.",n);
            return 0;
        }
    }
    return 0;
}
```